

Inference without Authority: the three authorities that govern an output

Alvaro Rivera

2026-07-20

Abstract

Every program produces output, and it is taken for granted that the code deciding what to emit also decides where it goes — to a database, a log, a topic, a screen. This paper argues that these are distinct authorities, and that a program speaking both with one voice has fused decisions belonging to different competences. Separating them reveals a third: what an output is — its projection — is no more the domain’s to shape than where it lands is the producer’s to name. An output is governed by three authorities — the domain, which knows what exists; the actor, which knows what becomes observable; and the assembler, which knows where observation occurs.

Their collapse has a signature. When one authority is made to pronounce another’s decision, it must reach a conclusion it has no standing to reach — an assertion beyond its warrant. This unwarranted assertion is the sibling, one level in, of the infrastructural symptom the series treats elsewhere: it compensates for an authority that is absent and dissolves when that authority is restored. The symptom appears at both ends of an output — a producer denied a voice for the destination commits to a where it cannot know; an observer given only snapshots infers a how it was never told — one defect, absent information, met once by stipulation and once by inference. Restoring the authorities dissolves both: the destination gains its own voice, and the record reaches the observer as an account it is told rather than one it must reconstruct. The observer comes to know by testimony, not by inference — within a trust boundary. Where the producer sits across one, as it does for anything leaving its own trust domain, testimony must be verified rather than simply believed; that boundary the paper marks as the bound of its scope, not a caveat to its result.

A journaled actor system serves as a worked instantiation in which the three authorities are kept apart — a demonstration that the separation is buildable, not a measurement of its cost or benefit. The contribution is analytic — a *theory for analyzing* in the sense of Gregor’s (2006, Type I): a criterion by which any architecture may be read, sound not by how it divides code, but by whether the authorities within it match the decisions each part has the standing to make. The labs are an existence proof of

realizability, not a design-science evaluation of cost or benefit.

Inference without Authority: the three authorities that govern an output

TL;DR

Every program has a `print`, and it is taken for granted that the code deciding *what* to emit also decides *where* it goes. These are distinct authorities. Pulled apart, they reveal a third: what an output *is* — its projection — is the actor’s to author, not the domain’s. An output is governed by three competences: the domain (what exists), the actor (what becomes observable), and the assembler (where observation occurs). Collapsing any two forces one authority to pronounce a decision it cannot know — an *assertion beyond warrant*, the paper’s construct, which sits at both ends of an output: a producer stipulates a *where* it cannot know; an observer given only snapshots *infers* a *how* it was never told (the genuine inference the title names). Restoring the authorities dissolves both. The contribution is analytic: an architecture is sound not by how it divides code, but by whether the authorities within it match the decisions each part has the standing to make.

Dependencies. This paper is part of the *Puppeteer Papers*, a series of self-deposited preprints, and rests on two of them: the actor’s speech and `tell` (Paper 4) and the journal as substrate (Paper 5), constructions established there rather than re-argued. Its symptom criterion is stated and justified here (§3), echoing — but not depending on — the one Paper 6 applied to infrastructure. Methodologically, it is an analytic theory contribution in the sense of Gregor’s (2006) theory for analyzing (Type I): the three authorities and the *assertion beyond warrant* are constructs by which an architecture is described and judged, while the labs of Appendix A are an existence proof of realizability, not a design-science evaluation of cost or benefit.

1. `print`

Every programming language has a `print` statement. It is so familiar that it has become invisible. A beginner learns that `print` displays text; a professional, that it writes to a log; a distributed-systems engineer, that it publishes to a topic, updates a read model, refreshes a cache, or emits telemetry. On what `print` does, everyone agrees. On a prior question — who is entitled to decide *where* its output goes — almost no one pauses.

This is not a paper about printing. It is a paper about authority over the destination of an output, and about what the familiar shape of `print` has quietly kept out of view.

Consider an accountant. She can specify, in complete detail, how the total of

a customer’s invoice is computed from an order: which lines are taxable, how discounts compose, how rounding is applied, when a line is excluded. Every rule of that projection is hers to state. Put to her a different question — should this invoice be written to MySQL or to PostgreSQL? delivered over a message topic, held in a cache, pushed to a dashboard? — and her answer is immediate, and correct: she has no idea. The destination is simply not something the computation of an invoice has an opinion about.

Yet business software is built, routinely, to answer exactly that question in the same place it computes the invoice. In a typical web service the whole of it is a single statement — `return Ok(invoice)` — and that one line names the total to show *and*, at the same stroke, fixes that the result is JSON, that it travels over HTTP, that it carries a 200, that it is synchronous. The module that knows how to total an order has also named the format, the protocol, the status, the timing. Two decisions — *what* becomes observable and *where* observation occurs — are folded into one body of code, and the fold is so customary that it goes unseen, in precisely the way `print` goes unseen.

The accountant sharpens the point because she is the producer of the observable and still cannot name its destination. This is not a limitation peculiar to accountants. It is a property of the relation between a computation and its output: the party that authors what becomes observable is not, in general, the party positioned to decide where observation occurs. The producer of an output can decide *what becomes observable*; it cannot decide *where observation occurs*.

If that is so, two questions follow, and the rest of this paper is their pursuit. First (§2–§4): if the producer does not decide the destination, who does — and what does that reveal about what an output actually *is*, once it is no longer mistaken for a write? Second (§5 onward): if the destination is bound elsewhere, how does an *observer* come to know what was made observable — and why does the conventional answer force the observer into an inference it should never have had to make?

2. The Second Voice

Section 1 ended with a separation. One party knows what should become observable — the invoice, its line items, its total. Another must decide where observation occurs — the database, the topic, the cache. The question is no longer whether these are different decisions; §1 has shown that they are. It is who is entitled to pronounce the second one.

The producer cannot simply acquire that standing. Nothing about how an invoice total is computed teaches where invoices are kept; the rules of the projection are silent on the choice of sink, and no refinement of them will make one speak it. The knowledge is not there to be found. The decision must therefore be pronounced somewhere else.

It is worth being exact about “somewhere else,” because the usual answers are

too weak. The destination is not merely computed in another layer, delegated to another module, or hidden behind another class. Those are the same speaker — one program, one author, arranging its own internals. What §1 forces is not a rearrangement of code but a difference of *standing* — and standing, here and throughout, is epistemic. It follows the ordinary norm of assertion: one may state as fact only what one is in a position to know. The producer’s want of standing to name the destination is therefore not a rule laid upon it; it is the plain consequence of a fact about its knowledge — the destination is not in it. The destination belongs to a different authority, and must be spoken in another voice; the authority is the ground, the voice only how it makes itself heard.

This series has met the notion of a voice before. Paper 4 established that an actor has one: it records its deeds in its own vocabulary, and may say only what it could genuinely have said. That discipline turned on the boundary of a single voice. The present separation is the discovery that, for output, a single voice is not enough — because a single voice carries a single competence, and the output answers to two. The actor can pronounce the projection: it lived the order, it knows the total. It cannot pronounce the destination: nothing it lived tells it where the total is kept. It has no standing to pronounce it.

This is the paper’s claim in one line. The actor speaks the projection; another authority speaks the destination. Where Paper 4 showed that the actor speaks, this paper shows that, for its output to be described correctly, the actor does not speak alone.

Because it turns on standing and not on mechanism, the separation is easy to counterfeit. A system can be split into layers, modules, classes, even two languages, and still speak with a single voice: splitting code does not split authority. A second module is not necessarily a second voice. What tells them apart is not whether there is another class, but who could author the second decision without modifying the first — for a second voice exists only when the second decision can be authored, replaced, and evolved independently, because it belongs to a different authority.

The principle outruns output. An architecture is not measured by how many layers it holds, but by whether the entitlements within it match the decisions each part has the standing to make. Measured so, most software has a single voice and pronounces the destination with it — what becomes observable and where, spoken as though they were one competence when they are two. That folding is so ordinary as to be invisible; §3 takes it up as what it is: a symptom.

3. The Symptom

Return to the collapse §2 ended on. When one voice is made to speak both what becomes observable and where, something has to fill the place where the second authority should have spoken. The voice has no knowledge of the destination — that was the whole point; nothing it lived tells it where its output is kept. Yet a destination is produced all the same. It has to be: the code must compile,

the write must resolve, the line must end somewhere. So the voice supplies what it cannot know. It *asserts* the destination — stipulating it, or taking a framework’s default — and lets that stand as fact.

That is the phenomenon, and it is worth being exact about it. The producer does not reason its way to MySQL; it weighs no evidence and concludes nothing. It settles on a destination and lets it stand — writes it, or inherits a default. The hardcoded sink is not a fact the producer possessed, nor one it inferred; it is a value it was forced to fabricate, standing in for an authority that was never given a voice.

This does not indict the producer’s other work, and the distinction is the whole of it. The projection *is* an inference, and a warranted one: the invoice total is derived, the subtotal computed, and the producer has the standing to derive them — it lived the order. What sits beside that warranted inference is not a second inference but a bare *assertion*: the destination, committed to as fact without the standing to know it. The symptom is not that a voice infers — inference, where warranted, is its proper work. It is that a voice asserts beyond its warrant.

None of this was derived from a prior construct. The assertion was forced by the argument of §1 and §2 — a fused authority, a decision it cannot know — before any name was reached for. What remains is to name it, and to say by what test the name is earned. Call a feature of a system a *symptom* when it bears two marks. It *compensates*: it stands in for something absent, doing the work of a capability the system does not have. And it *dissolves*: supply the missing capability and the feature vanishes, with nothing left for it to do. Together the marks separate what a problem genuinely demands from what a deficiency elsewhere induced — a feature the problem itself requires would persist even in an otherwise ideal system, while one that disappears the moment some other lack is repaired was never answering the problem, only the lack. Dissolution-on-repair is the diagnostic: the counterfactual that tells an induced feature from an intrinsic one.

The unwarranted assertion bears both marks exactly, now about a decision rather than a component. It compensates: the producer cannot know the destination, so the assertion stands in for the competence that could — it is what the single voice says because a second voice was not there to say it. And it dissolves: give the destination its own authority, let a second voice pronounce it, and the assertion simply disappears; there is nothing left for it to compensate for. What looked like a decision the producer had to make turns out to be an artifact of who was allowed to speak. That is what earns the name. The symptom is an **assertion beyond warrant**: a value a voice commits to as fact only because it was made to pronounce a decision it had no standing to know. (The same test recurs across the series: Paper 6 applied these two marks to infrastructure layers — a cache, an object-relational mapper, a queue — asking whether each existed because the problem demanded it or because the persistence model was deficient. Here the object is a decision, one level in, rather than a component;

the criterion is the one just stated, not one borrowed to be believed on another paper’s word.) Its most literal form — a genuine inference — waits at the other end of the output, in §5; that is where the word *this paper* is named for fits without strain.

This is also what §2’s criterion looks like from the inside. An architecture is sound when its authorities match its decisions; the symptom is the moment they do not — when a decision is pronounced by a voice with no claim to it, and the gap between the two is papered over by an assertion nothing warrants. Read this way, a hardcoded destination is not a small infelicity of style. It is the visible trace of a missing authority.

Which raises the question a reader has been holding since §2. If the separation is forced by the argument, and its absence leaves a mark this legible — does any system actually honor it? §4 answers.

4. The Three Authorities

By §3 the argument has separated two authorities: the actor, which pronounces the projection, and the assembler, which pronounces the destination. But the projection has a boundary on its other side as well, and naming it completes the structure.

The actor authors the projection — but not out of nothing. It authors it over a material it did not invent: the objects, the state, the operations that make an order an order. That material is the domain’s. The domain knows what exists; the actor knows only what, of what exists, becomes observable. These are not one competence, and the distance between them shows itself the moment a domain object renders itself: in doing so the domain has reached past *what exists* into *what becomes observable* and taken the projection, which was never its to take. The symptom there is not that the object prints. It is that the material spoke as though it held the projection’s authority.

Three competences, then, not two — and the series has been uncovering them in turn. Paper 4 found that the actor may not name the *transport* that carries its voice to a peer. This paper found that it may not name the *sink* where its voice lands for an observer. And now a third, pointing the other way: the domain may not author the *projection* — what becomes observable belongs to the actor, not to the material. The actor stands in the middle, its competence bounded on both sides:

Authority	Knows	Governs
Domain	what exists	the material
Actor	what becomes observable	the projection
Assembler	where observation occurs	the destination

Table 1. The three authorities that govern an output.

what exists (material)		what becomes observable (projection)		where observation occurs (destination)
+-----+ A		+-----+ B		+-----+
DOMAIN -----		ACTOR -----		ASSEMBLER
+-----+		+-----+		+-----+
edge A (domain actor) : SOFT -- negotiable per concept				
edge B (actor assembler) : HARD -- the destination is never in the producer				
a symptom is a competence reaching ACROSS its edge:				
across A : an object that renders itself (ToString) [domain -				
> projection]				
across B : a hardcoded sink or inherited default [actor -				
> destination]				

Figure 1. The actor bounded on both sides — a soft, negotiable edge to the domain and a hard, fixed edge to the assembler — with each symptom (§3-§4) marked as a reach across an edge into a decision the reaching party has no standing to make.

Read against this, the two symptoms are one kind of thing: a competence reaching across a boundary. A domain object that renders itself is the domain reaching up into the projection; a hardcoded sink is the actor reaching up into the destination. In each, one authority pronounces a decision that belonged to another and papers the gap with an assertion it has no standing to make. This is not a stack of layers — layers stack the same speaker — but a division of what each party is in a position to know.

A realization of the structure follows directly from it: give each competence its own place to speak, and the symptoms have nowhere to form. A reader who has come this far is right to ask whether anything actually builds it. One system does. Puppeteer is one such realization, and — because it was not built to make this paper’s point — the separation it exhibits was already in place before the point was named: found, not staged. That is the whole of the claim, and it is a demonstration of buildability, not a measurement. Three labs on the running framework (Appendix A) make it concrete: the same actor and the same projection reach genuinely different destinations — real SQL Server and MySQL among them — with no edit to the producer; one journaled fact is observed by three distinct views without a single new domain method; and an observer reaches the same knowledge whether it is told or polls. Each is reported against a fused baseline, which pays one edit — one producer change, one domain method — for what the separation gives for nothing. What the labs do not do is measure: no cost against that baseline at scale and no benefit in production, which are left to other work. The three authorities are the

argument's, not the framework's.

In it, a program is written across two languages. The domain — an **Order** and its **OrderItems**, the operations each genuinely performs — is written in a general-purpose host language, and it holds *what exists*. An actor is a journaled unit: its state is not stored and mutated in place but reconstructed by replaying the operations it has recorded (Papers 1–2; the journal of Paper 5), so it has a *history* — the journal of those operations — and, at any moment, a *state* — the object graph that history reconstructs. What the actor does when a command reaches it is written in a small DSL — it declares, branches with **if**, walks with **foreach**, and emits with **print** and its durable form **expose** — and what it authors is *what becomes observable*: it walks its state (the order's items, each item's product name and its subtotal — unit price times units), selects the primitive leaves the projection needs, and emits them, naming a logical output and no destination at all.

In the DSL the projection is a walk that narrows and emits, and where it narrows is a boundary of its own. Over Microsoft's **dotnet/eShop Order** aggregate — the same port the systems papers use — an invoice over discounted lines reads:

```
foreach (item in order.OrderItems) {  
    if (item.Discount > 0) {  
        print item.ProductName product,  
              item.UnitPrice * item.Units subtotal;  
    }  
}
```

Every name is the domain's own — **OrderItems**, **ProductName**, **UnitPrice**, **Units**, **Discount** — and the subtotal is computed on the way out (**UnitPrice * Units**), a derivation, not a stored field. The narrowing sits in the actor's **if**; it could instead be pushed into the material, the domain exposing the narrowed collection as a verb of its own (**order.DiscountedItems()**), which is legitimate only when *discounted items* is a notion the domain could genuinely own — worth shaping its write representation for. That shaping is not hypothetical here: the eShop item already copies its product's name onto itself rather than pointing at a catalog, so a line can be read without leaving it. Pushing a selection down is the same trade; making it for a view the domain does not own is the symptom. Either way, **print** emits values under names and names no place. The assembler, outside the actor, binds that logical output to *where observation occurs*: a table in MySQL, a topic in Kafka, a hub, a file, the console, nothing — and whether it is pulled or pushed. That the script is unchanged either way is not asserted here only rhetorically: the framework's own output interface documents it — *pull versus push is a property of the destination, never of print* — with the sink configured outside the actor, at the hosting handler, identically across every topology.¹ The shape is Unix's **>**, with the modern range of sinks in place of a file.

And the shell's other operator has a counterpart too. Where **>** redirects a

program’s output to a sink, `|` pipes it to another program; Puppeteer’s `|` is the `tell` of Paper 4 — the same projection sent not to a store but to a peer actor, received as a told message. The two are distinct planes, not one stream routed a single way, so an actor’s output can want both operators at once — `program params > target | peer params`, redirected to a sink *and* piped to a peer. No shell grants that in one stream; it makes you choose `> file` or `| program`. Puppeteer does not, because the destination is the assembler’s to bind and the pipe is Paper 4’s `tell` to carry — and the projection in the middle, as before, learns of neither.

Put §2’s test to it. Who could send the invoice to Kafka instead of MySQL without modifying the actor? The assembler alone; the actor is not touched, not recompiled, not reread. The same actor that writes to a warehouse in production will, run from a command line with its output bound to the console, print the same invoice to the terminal, where it can be piped into the ordinary tools. Two destinations, one projection, and the projection learned of neither. That independence is the operational face of the separation: each competence can be authored and revised without leave from the others.

These two edges are not equally hard, and the structure is more honest for saying so. The actor/assembler boundary does not move: no refinement of how a total is computed will yield where totals are kept, so the destination is never the producer’s to name — the knowledge is simply absent from it. The domain/actor boundary is not fixed in that way. Whether *discounted items* is *what exists* — a notion the domain genuinely owns, worth a place in its model — or only *what becomes observable* — a narrowing the actor imposes for a view — is a modeling judgment, and for a given concept it can fall either way without anyone overreaching; that is why the narrowing above could be pushed into an `order.DiscountedItems()` as a legitimate alternative rather than a mistake. The line between the material and the projection is drawn per concept; the line between the projection and the destination is not drawn but discovered. What does not move is the rule that governs both once a line is set: neither side may reach across it — a domain object that renders itself has reached past whatever it models as *what exists* into a projection it never chose to own, and that is the symptom wherever the boundary sits. Table 1 names the three authorities and Figure 1 draws their two boundaries — but the two are not edges of one hardness: one is a fact about absent knowledge, the other a judgment about owned knowledge. The table lists the competences; the figure marks that their boundaries are not cut equally deep.

There is a way to see the hard edge without arguing it: test for it. If the destination were the producer’s, no test of the projection could run without also standing up the sink — to exercise what the actor computes you would first have to say where its output goes. The opposite is what holds. The projection can be exercised end to end — the order placed, the invoice totalled, the result read back — with no destination bound at all, on the pull path alone: the answer returns to the caller and nothing is sent anywhere. That a test of the domain

needs no sink is not a convenience the framework grants; it is the operational proof that the sink was never in the domain to supply. This is the epistemic claim made verifiable, and falsifiable in the plainest sense the word allows: were the destination truly the producer's, no such test could pass without it. Here the analysis of §4 meets something checkable — the labs of Appendix A run exactly this test, exercising the projection while the destination stays a separate, unbound choice.

One primitive has been named but not yet used. Where `print` makes a projection observable in the moment, `expose` persists it into the actor's history — the projection enters the record the actor keeps, durable and replayable. That difference is the hinge of §5, because it is the difference between an observation that merely passes and one that enters a narrative an observer can later be told.

Puppeteer is offered as *a* realization, not *the* realization. The paper does not claim this DSL is necessary; §2 was explicit that each authority may be housed in any language, or in none. The claim is that the three authorities are real, that they can be kept apart, and that where they are, the material, the projection, and the destination each leave the others' voice. That one working case shows the separation is not only arguable but built — and what it opens for the party on the receiving end, the observer, is §5.

5. The Observer

Every section so far has stood beside the producer. Turn now to the party on the other end — the one that receives what was made observable. Call it the observer: a dashboard, a report, a downstream service, a screen. There is a question about the observer this paper does not take up — how it *interprets* what it receives, reading a sequence of acts for the routines they compose. The question here is narrower, and prior: how does the observer come to *know* the state at all, and does the symptom of §3 reach it too? It does — and its situation turns out to mirror the producer's exactly.

An observer comes to know the state by one of two means. It can read a *snapshot* — the configuration as it stands right now: this invoice, this total, this set of open orders. Or it can receive a *narration* — the sequence of operations that brought the state about: the order placed, the line added, the discount applied, the invoice settled. The two are not interchangeable. Both yield the same configuration; only the narration preserves how the configuration came to be. A snapshot is an answer with its reasons erased.

Now give the observer only snapshots, and ask of it anything the snapshot does not carry — how the total was reached, what has changed since it last looked, why an order stands where it does. It cannot read the answer off; the snapshot does not hold it. Yet it must answer all the same, and so it does what the producer did in §3: it reconstructs what it was not given. It diffs successive snapshots to guess what changed; it re-derives the path from the endpoint; it

polls, and polls again, rebuilding a history from a series of stills. The observer infers the history it was not given.

And this is the same defect, at the far end of the voice — but here, at last, it takes the form of a genuine inference. The history was never the observer’s to know; it belonged to the actor, which lived it. Reconstructing it from snapshots is an assertion reached beyond the observer’s standing, exactly as the stipulated sink was — except that the observer truly *infers* it, diffing and re-deriving, where the producer merely stipulated. **This is the inference the paper is named for:** not the producer’s quiet stipulation, but the observer’s laborious reconstruction of what it was never told. What §3 found at the point of emission recurs at the point of reception: the producer asserted a *where* it could not know; the observer infers a *how* it was never given. One defect — assertion beyond warrant — book-ending the output, met once by stipulation and once by inference.

The polling is not the observer’s failing; it is forced, in the same way the producer’s stipulation was forced. An observer handed a snapshot and left to rebuild the rest has been given the configuration but not the account behind it — a boundary that shows the answer and withholds its reasons. Its polling is the mechanical signature of a missing narration.

The dissolution is the one already seen, read from the other side. The earlier papers built a substrate in which the record is not a snapshot but an account: the journal (Paper 5) is the sequence of operations the actor lived, and each entry is something the actor could genuinely have said (Paper 4). Deliver that to the observer — the record the actor keeps, into which **expose** enters the observations meant to last — and the observer has nothing left to reconstruct. It is not diffing stills; it is being told.

What the observer has then is *testimony*: it comes to know the state the way anyone comes to know a fact they did not witness — on the word of the party that did. And this names the shape of the whole paper. The two — the producer’s stipulated *where* and the observer’s inferred *how* — were born in one place: absent information. The producer was not given the destination, and asserted one; the observer was not given the account, and reconstructed it. Neither gap is closed by a mechanism. Each is closed by a kind of knowledge delivered to the party that lacked it — the destination, owned by a second authority; the account, carried by the actor’s own record. The observer stops being an inference engine; it still computes the view it needs, but over an account it was given, not a reconstruction it was forced to fabricate. Spoken in full and received as testimony, the voice leaves nothing at either end for an unwarranted assertion to fill.

This resolution has a boundary, and in distributed practice not a small one. Testimony is knowledge on another’s word, and only as strong as the standing of the one who gives it. Within a trust boundary — producer and observer under one authority, no adversary between them — the account can be taken as given, and the observer is spared its reconstruction. Across a trust boundary,

which for anything leaving its own trust domain is the ordinary condition rather than the exception, the observer cannot simply believe what it is told: it must verify, and verification is a need this paper names but does not meet (§6). The dissolution just reached is therefore real but scoped — it frees the observer from rebuilding a *withheld* history, not from checking an *untrusted* one.

Paper 4 gave the actor speech; this paper lets the observer receive it as testimony. But a narration received is not yet a narrative recognized — what the observer makes of the account it is told, reading a sequence of acts as the routines the actor performed, is a further question, beyond the scope of this paper.

6. Limits

An argument is only as trustworthy as the boundary it draws around itself, the more so when it ends on a dissolution — the point at which it is easiest to claim more than was shown.

The hardest limit follows from the paper’s own root. The symptom was traced to absent information: a party forced to act on knowledge it was not given fills the gap with an assertion nothing warrants, and restoring the authority that holds the knowledge makes it vanish. But that cure works only where the absence is *architectural* — where the knowledge exists, held by some party, and was merely withheld by a collapsed boundary. It does nothing for an absence imposed by the world. An observer that must not only read but *write* while cut off from the account is denied the state by physics, not by architecture; its speculation about what it cannot see is genuine, and no rearrangement of authorities abolishes it. §5 frees the observer from reconstructing a history that was withheld; it does not free a writer from guessing at a history that has not reached it. The three authorities dissolve absent-information-by-design; they leave untouched the real difficulty of absent-information-by-disconnection.

A second limit lies in the word the paper ends on. Testimony is knowledge on the word of another, and it is only as good as the standing of the one who gives it. The observer is spared inference precisely because it trusts the source — the account reaches it as something the actor could genuinely have said. And across a trust boundary that assumption usually fails — between organizations, over a public network, anywhere the producer is not under the observer’s own authority — so the untrusted producer is the default case in distributed systems, not an edge of it. There an adversarial or unreliable producer makes testimony insufficient, and the observer needs verification, a different question this paper does not take up. This is the boundary of the result’s scope, not a caveat within it: the argument resolves absent-information inside a trust domain and stops where trust does.

Nor does the paper solve delivery. It has said that the assembler binds the destination and carries the account to the observer; it has not said how the account survives a partition, or reaches a thousand observers rather than one. Those are questions of transport, answered by the substrate the earlier papers

built, not by the division of authorities argued here. Fan-out scales with the transport, not with the journal — a slow or lossy channel is a real constraint, and an orthogonal one.

And the symptom is conditional on the knowledge being absent — worth stating plainly, because it bounds the claim. Where a single hand genuinely holds both knowledges — a sink fixed and known at authoring time, in a program that will only ever deploy one way — naming the destination in the producer asserts nothing it cannot back, because nothing was unknown; there is no symptom, only a coupling. What separation buys there is not correctness but evolvability: the freedom to change the sink without editing the producer. The epistemic claim is the stronger one, and it bites wherever the two knowledges genuinely part — most of production software, but not all of it.

Finally, the structure earns its keep only where there is a second end for it to have. In a domain where the party that produces the output and the party that observes it are one — a single-player program with no downstream reader — the symmetry has nothing to bite on, and the apparatus is overhead. The claim is not that every output has three separate authorities in practice, but that where the roles are genuinely distinct, so are the competences, and collapsing them leaves a mark.

A final scope, and it is the widest. This paper argues a single axis — the output — and demonstrates the three authorities only there. The choice is not arbitrary: the output is the axis whose boundary is **hard**. The destination is simply not in the producer, and no refinement puts it there, so the separation shows without noise. Other axes submit to the same authority *question* — what the world speaks in, the clock a computation reads, the draw it makes — but their boundaries are subtler, because a capture on those axes must decide *what* to take and *when*, where **print** decides nothing of the sort. This paper does not settle them; it claims the output alone, as the first and cleanest axis of a principle, not the whole of one. To read it as a promise of total portability would be to credit it with work it has not done — and to invite exactly the question it does not answer.

7. Related Work

The idea that a program should not name where its output goes is not new; what is new is reading it as a matter of authority. The mechanism has a long lineage. Unix bound the destination outside the program — a process writes to a file descriptor and the shell decides, through `>` and `|`, whether that descriptor is a terminal, a file, or another program (Ritchie & Thompson, 1974); Plan 9 pushed the same indirection into per-process name spaces (Pike et al., 1992); flow-based programming (Morrison, 1994) and Kahn process networks (Kahn, 1974) gave components ports and let an external topology decide what connects to what. All of these late-bind the destination. None asks whether the program was *entitled* to name it; the binding is offered as composition, not as competence.

Three lines come closer, and each holds one edge of the argument without its reframe. Coordination languages (Gelernter & Carriero, 1992) split a program into a computation model and a coordination model — what it computes apart from how it is connected — which is the material/assembler distinction in all but name, drawn as modularity, with no epistemology attached. The object-capability model (Miller, 2006) is built entirely on authority — no ambient authority, least privilege, the right to act conferred rather than assumed — but turns that authority on access and effect, not on the projection or destination of an output. One line holds the split, the other the authority; neither turns either on output as an epistemic act.

The third line is the sharpest test, and the paper’s novelty stands or falls on it. Algebraic effects and their handlers (Plotkin & Pretnar, 2009) give the inversion its most rigorous form: a computation *performs* an operation — `print` among them — without fixing its interpretation, and a handler installed by an enclosing context decides what becomes of it, covering both directions of the boundary with a semantics rather than a convention. Were the contribution only *that the destination is bound elsewhere*, effects would already hold it whole, and this paper would be late binding with a story attached. The delta is that the effect discipline is a mechanism and the claim here is a criterion that judges the mechanism’s use. An effect system relocates *where* an interpretation is written; it says nothing of whether the party writing it had the standing to know the value it supplies. Nothing in it stops a handler from hardcoding MySQL — the stipulation of §3, now lodged inside the handler — and the program stays perfectly well-typed, because effect typing tracks *which operations are handled*, a matter of control and safety, never whether a handler was entitled to the value it names. Effect-safety and authority-soundness are therefore independent: a computation can be fully effect-safe and still carry the symptom, which lives on a different axis — not who *intercepts* an operation, but who is in a position to *know* its value. This is the epistemic-versus-compositional distinction the paper turns on, and it is load-bearing, not cosmetic: withdraw it and nothing lets one call an effect-handled `print` with a fixed sink defective, for by the calculus’s own lights it is not. The epistemic reading is not a gloss on late binding; it is the axis along which a program that already uses effects is graded.

Two structural gaps confirm the boundary is not the same one. Effects split an operation from its interpretation — a binary — where this paper finds three authorities; and the finest of the three, the domain’s inability to author the projection (the self-rendering object of §4), has no counterpart in an effect, whose handled value the producer computes with no sense that its *shape* answers to a different competence than the *material* it is drawn from. And effects are producer-side throughout: they say nothing of how an *observer* comes to know a state, and so leave untouched the second half of the argument — the snapshot-versus-narration symptom of §5, where the same defect returns with no operation to handle. The split, the inversion, the authority: each neighbouring line holds one, none holds all three, and none reads output as an epistemic act.

Closest, and worth separating out, is the revised Single Responsibility Principle, which asks that a module answer to a single *actor* — a stakeholder with the authority to request change (Martin, 2017, ch. 7). This is separation by authority, the very criterion argued here — but its authority is organizational: who may *ask* for a change. The authority of this paper is epistemic: who is in a position to *know*. The producer does not withhold the destination out of good manners toward another team; it cannot supply it, because the knowledge is not in the projection. That distinction is what separates the present claim from the ordinary counsel of layered architecture. Controllers and views, services and DTOs, records — these divide technologies and structure, not authorities. When a controller returns a view, a DTO, a file, the same code that chose what to show has also fixed that it is HTTP, and JSON, and synchronous; the DTO does not defer the destination but freezes the projection into a value, and changing the sink recompiles the producer.

The objection nearest to a practitioner is that this is dependency injection, or hexagonal architecture — ports and adapters — renamed. It is best met on its own ground, which is the test suite, not philosophy. Hexagonal inverts the dependency: the domain declares a *port*, an output interface, and an adapter chosen from outside implements it. That is genuine late binding — but a port is still something the domain names and depends upon, and to test the domain one must supply a double for it, a fake sink standing in for the real. This separation leaves no port. `print` depends on no output interface, not even an abstract one; it emits values under a logical name and knows of no destination to invert. The difference is observable, not interpretive: count the doubles a domain test must stand up for its output — under hexagonal, at least one; here, none, because there is nothing there to double (§4). Inversion relocates a dependency; this removes it, and that is the line injection cannot cross, because injection presupposes a thing to inject.

A contemporary preprint approaches the same vocabulary from the opposite direction, arguing that tool boundaries do not confer epistemic warrant on what an agent takes *in* as observation (Romanchuk & Bondar, 2026). Where it asks what an agent is entitled to believe from its inputs, this paper asks what a producer is entitled to decide about its outputs; the shared words — warrant, epistemic — mark an adjacency, not an overlap. The intuition beneath that adjacency is not itself new: information-flow control has long held that a value’s classification is not laundered by crossing a boundary — a label follows the data through whatever module it passes (Denning, 1976; Sabelfeld & Myers, 2003) — and the same instinct animates data provenance. What the neighbouring preprint adds is the agent-era statement of it; what steadies the positioning here is the older observation, so the neighbourhood does not rest on a single recent source.

Two long-standing models sit across from the claim rather than beside it. Communicating sequential processes (Hoare, 1978) and the actor model (Hewitt et al., 1973) both name the recipient of a message explicitly — a channel, a mail

address. That explicitness is precisely what the present argument identifies as the reintroduced coupling: to name the party at the other end is to take, in the sender’s voice, a decision the sender has no standing to make.

What remains, after all of these, is an unclaimed intersection. The mechanism of late binding is old; the authority framing exists for access; the coordination split exists for modularity; the effect inversion exists for semantics. But output read as an epistemic boundary — a producer entitled to what becomes observable and not to where it lands, the naming of a destination as an inference reached beyond warrant — is a description the literature has left open. The neighbouring terms are already taken: *observability* by telemetry, *output as a boundary* by information-flow security, **print**-versus-logging by engineering hygiene. The reading offered here fills a gap it did not have to invent.

8. Conclusion

This paper began at the most familiar instruction in programming — **print** — and found beneath it, where everyone thought the matter settled, a question no one was asking: who is entitled to decide where an output goes. Pulling on it opened not one hidden decision but three. What exists, what of it becomes observable, and where observation occurs are separate competences, held by separate authorities — the domain, the actor, and the assembler of Table 1.

The construct followed from the division. Wherever one authority is made to pronounce another’s decision, it must reach a conclusion it has no standing to reach — an assertion beyond its warrant — and such an assertion is a symptom in the sense §3 defined: it compensates for an authority that is absent, and dissolves the moment that authority is restored. The symptom proved to sit at both ends of the output. The producer, denied a voice for the destination, stipulates a *where* it cannot know; the observer, denied the account, infers a *how* it was never told. One defect — absent information — met once by stipulation and once by inference.

What dissolves both is not a mechanism but a restoration of authority. Give the destination its own voice, and the producer stops asserting a *where* it cannot know. Deliver the record as what it is — the sequence the actor lived, an account it could genuinely have given — and the observer stops reconstructing how the state arose. It returns from being an engine of reconstruction to being, simply, a reader of what it was told.

None of this is a fact about **print**, or about any one system. The three authorities are a way of reading any architecture. The usual measure — how many layers, how clean the separation of code — misses what matters. The measure that holds is the one this paper has applied throughout: an architecture is sound not by its divisions of code but by whether the authorities within it match the decisions each part has the standing to make. That is a test a reader can carry back to a system already built and run against it — as Paper 6 offered a test for infrastructure, this offers one for the authorship of output.

Set against the series, the result is a mirror, and it has a spine of three words, one to a paper. Paper 4 showed that the actor *speaks*. This paper shows that it does not speak alone, and that what it speaks reaches the far end as *testimony* — received, where that word can be trusted, on the word of the one who lived it, not reconstructed by the one who did not. What testimony becomes when it is finally read — its *narrative* — the series has still to take up. Each of these makes one fundamental artifact explicit, and together they are the material of a synthesis this series is working toward, not the business of any single paper. Here it is enough to have shown that an output has authorities, that they can be kept apart, and that when they are, no one at either end of the voice is left to infer what someone else was in a position to say — within the trust domain where that voice can be believed, the observer, at the last, is simply told. Where it cannot, verification remains, and that frontier is the paper’s edge, not its undoing.

Appendix A. Labs

The three authorities of §4 are exhibited here on the running framework. These labs are illustration, not measurement: each demonstrates that a separation the argument reached analytically is *buildable*, and that it holds mechanically against a *fused baseline* — the ordinary arrangement in which the two decisions are written together. None measures cost against that baseline at scale, or benefit in production; those are the questions §6 leaves open. In the manner of the substrate paper’s labs, each result is a count the separation drives to zero — the zeros are the claim.

Lab A — the destination (the assembler’s authority). One actor records an order and projects it with a single `print` script; the projection is then bound, from outside the actor, to one destination after another — a real SQL Server table, then a real MySQL table — changing only the writer. Both backends receive identical rows, and the actor is neither recompiled nor reread. The wire format (TOON or JSON) and the direction (pull or push) are bound in the same outside place. A fused baseline — the projection and the sink written together — pays one producer edit for each new destination; the separated actor pays none. And the pull test makes the epistemic point empirically: the projected total is read back through a query with nothing pushed — a domain result exercised while no destination takes part, which is the operational proof that the sink was never in the domain (§4).

Lab B — the projection (the observer’s authority: what). From one journaled fact, three distinct observers are added — a fulfilment view, a finance view whose figure is *derived in the projection* (unit price $\times$ units), and a catalog view — each a projection reaction over the same fact, each authored without adding a single method to the domain. The domain’s surface is unchanged, confirmed by reflection; a fused baseline pays one domain method per view.

Lab C — the direction (the observer’s authority: how). The same fact

reaches an observer by two routes: it is *told* — a reaction carries it across a **tell** to the observing role — or the observer *polls* it with a query. Both deliver the identical observation, and the producer’s domain holds no method for either direction: being told is a reaction, polling is a query, and neither is the domain’s to name.

Lab D — testability (the hard boundary is observable). The separation of §4 is not only argued; it can be tested for. A domain projection is exercised end to end — an order recorded, its total read back — with no destination bound at all: no sink, no port, no test double for output. That the test needs none is the operational proof that the sink was never in the domain; were it the producer’s, the test could not run without it. Against a hexagonal (ports-and-adapters) baseline the difference is a count: an output test of the ported domain must stand up at least one double — the port’s — while the separated domain stands up zero, because there is no port to double.

Lab	Separation	Separated cost	Fused baseline
A	destination bound outside the actor (real SQL Server, MySQL)	0 producer edits per destination	1 producer edit per destination
B	N views over one fact	0 new domain methods for 3 views	3 domain methods
C	told vs polls	0 domain methods for the direction	no fused analog
D	domain output tested with no destination	0 test doubles for output	≥ 1 (hexagonal port)

The labs live in the framework’s test suite; the real-backend case runs against containers and self-excludes from the default run. They show the separation is not only arguable but built, and that its mechanical benefit against a fused baseline is a set of edits not made — leaving the economic question, whether those unmade edits are worth the apparatus, to §6 and to future work.

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit [0bf947b](#) (2026-07-22). The snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:4cce51c877c0836f5a561ce2d25bca9f7800ee71;  
  origin=https://github.com/alvaroNCubo/puppeteer;  
  anchor=swh:1:rev:0bf947bd6563e34cb141e3b5ba6cd13b4a811023
```

The three source files this paper names — `Puppeteer/IOutputSink.cs`, `Puppeteer/EventSourcing/Follower/Planes.cs`, and `Puppeteer-Cli/AttachCommand.cs` — resolve against this snapshot. A reader can construct a per-file SWHID by adding the qualifier `;path=<path>` to the directory SWHID above. Future commits to the repository may move these files; the SWHID preserves the cited state independently of any future change to the repository or its hosting.

The four laboratories’ source and datasets accompany this paper as `paper08-data.zip`. Each lab’s result table — the counts this paper cites in Appendix A — is included in full; the labs are count-based, so no large raw sample log is produced or omitted.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author’s.

Notes

¹ `Puppeteer/IOutputSink.cs` (interface `IOutputSink`, struct `PushDocument`). Its documentation comment states the separation directly: “*Pull vs push is a property of the destination, never of `print`. The DSL script is identical either way... the sink is assembly-agnostic: it is set via `OutputTarget(sink)`... because the mechanism lives at the `ActorHandler`, not in any one topology.*” The four output planes an actor’s reaction can address — `Program.Emit` (ephemeral projection), `Causation.Continue` (`tell`), `Outbox` (durable, exactly-once-recorded), `Metadata` (elide / materialize) — are defined in `Puppeteer/EventSourcing/Follower/Planes.cs`. The console binding referenced in §4 is `puppeteer attach`, whose REPL runs the actor with its output on `Console.Out` (`PuppeteerCli/AttachCommand.cs`), pipeable into ordinary shell tools.

References

- Denning, D. E. (1976). A lattice model of secure information flow. *Communications of the ACM*, 19(5), 236–243. <https://doi.org/10.1145/360051.360056>
- Gelernter, D., & Carriero, N. (1992). Coordination languages and their significance. *Communications of the ACM*, 35(2), 96–107. <https://doi.org/10.1145/129630.129635>

- Gregor, S. (2006). The nature of theory in information systems. *MIS Quarterly*, 30(3), 611–642.
- Hewitt, C., Bishop, P., & Steiger, R. (1973). A universal modular ACTOR formalism for artificial intelligence. *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, 235–245.
- Hoare, C. A. R. (1978). Communicating sequential processes. *Communications of the ACM*, 21(8), 666–677. <https://doi.org/10.1145/359576.359585>
- Kahn, G. (1974). The semantics of a simple language for parallel programming. In J. L. Rosenfeld (Ed.), *Information Processing 74: Proceedings of the IFIP Congress* (pp. 471–475). North-Holland.
- Martin, R. C. (2017). *Clean architecture: A craftsman’s guide to software structure and design*. Prentice Hall.
- Miller, M. S. (2006). *Robust composition: Towards a unified approach to access control and concurrency control* [Doctoral dissertation, Johns Hopkins University].
- Morrison, J. P. (1994). *Flow-based programming: A new approach to application development*. Van Nostrand Reinhold.
- Pike, R., Presotto, D., Thompson, K., Trickey, H., & Winterbottom, P. (1992). The use of name spaces in Plan 9. *Proceedings of the 5th ACM SIGOPS European Workshop*, 72–76.
- Plotkin, G., & Pretnar, M. (2009). Handlers of algebraic effects. In G. Castagna (Ed.), *Programming Languages and Systems (ESOP 2009)*, Lecture Notes in Computer Science (Vol. 5502, pp. 80–94). Springer. https://doi.org/10.1007/978-3-642-00590-9_7
- Ritchie, D. M., & Thompson, K. (1974). The UNIX time-sharing system. *Communications of the ACM*, 17(7), 365–375. <https://doi.org/10.1145/361011.361061>
- Rivera, A. (2026d). Preserving semantic continuity across actors: a tell-based approach without orchestration. *Puppeteer Papers Series*, Paper 4 [Preprint]. Zenodo. <https://doi.org/10.5281/zenodo.21207062>
- Rivera, A. (2026e). The journal as substrate: unifying deployment, replication, backup, and offline operation in distributed systems. *Puppeteer Papers Series*, Paper 5 [Preprint]. Zenodo. <https://doi.org/10.5281/zenodo.21349146>
- Rivera, A. (2026f). Most infrastructure layers are symptoms of the persistence model: a construct for auditing production stacks. *Puppeteer Papers Series*, Paper 6 [Preprint]. Zenodo. <https://doi.org/10.5281/zenodo.20317450>
- Romanchuk, O., & Bondar, R. (2026). Semantic laundering in AI agent architectures: Why tool boundaries do not confer epistemic warrant. *arXiv preprint arXiv:2601.08333*.

Sabelfeld, A., & Myers, A. C. (2003). Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1), 5–19. <https://doi.org/10.1109/JSAC.2002.806121>